

Deep Learning

```
In [1]: import pandas as pd
import numpy as np
from torch import nn
from torch.utils.data import DataLoader, Dataset, TensorDataset
from torchvision import datasets
from torchvision.transforms import ToTensor
import torch
import torch.nn as nn
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
from torchinfo import summary
from torch.utils.tensorboard import SummaryWriter
from torch.utils.data import WeightedRandomSampler
import os
from datetime import datetime
# counter
from collections import Counter

from notifier import send_notification, send_image, send_email
import time
from typing import Dict, Optional, Any

import ray
from ray import tune
from ray.tune.search import ConcurrencyLimiter
from ray.tune.search.optuna import OptunaSearch
from ray.tune.schedulers import ASHAScheduler
```

```
In [2]: writer = SummaryWriter()
```

```
In [3]: with open('../data/processed/dataset.pkl', 'rb') as f:
dataset = pickle.load(f)
```

```
In [4]: device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
device
```

Out[4]: 'mps'

```
In [5]: dataset.keys()
```

Out[5]: dict_keys(['X_train', 'y_train', 'X_val', 'y_val', 'X_test', 'y_test', 'X_train_store_category_le', 'X_val_store_category_le', 'X_test_store_category_le', 'y_scaler', 'X_scaler'])

```
In [6]: X_train = dataset['X_train'].drop(columns=["store_primary_category"], axis=1)
y_train = dataset['y_train']
X_test = dataset['X_test'].drop(columns=["store_primary_category"], axis=1)
y_test = dataset['y_test']
X_val = dataset['X_val'].drop(columns=["store_primary_category"], axis=1)
y_val = dataset['y_val']
X_train_store_category_le=dataset['X_train_store_category_le']
X_val_store_category_le=dataset['X_val_store_category_le']
X_test_store_category_le=dataset['X_test_store_category_le']
y_scaler = dataset['y_scaler']
```

```
In [7]: len(X_train_store_category_le.unique())
```

Out[7]: 66

```
In [8]: X_train
```

Out[8]:

	total_items	subtotal	num_distinct_items	min_item_price	max_item_price	total_onshift_partners	total_busy_partners	total_outstanding_orders	created_at_day	created_at_week_of_month	...	order
0	-0.887847	-0.774580	-1.034172	1.175252	0.233244	0.356917	-0.239323	-0.063258	-1.134736	-1.239225	...	
1	0.323576	0.936110	0.203869	-0.179647	0.466436	-0.898395	-0.866322	-0.691474	0.050511	0.223960	...	
2	0.727384	0.600475	0.822890	-0.664373	0.053865	-0.985975	-1.023072	-0.938953	0.912508	0.955552	...	
3	-0.080232	-1.258467	0.203869	-1.151047	-1.817052	-0.927588	-1.023072	-0.938953	0.697009	0.955552	...	
4	1.938807	1.653122	2.060932	-1.053712	-0.295923	1.115942	0.857927	1.078952	1.666756	1.687144	...	
...	...	...	...	...	...	...	...	...	...	...	...	...
124094	-0.080232	-0.288488	0.203869	-0.179647	-0.835852	-0.256143	-0.803622	-0.653400	1.451257	1.687144	...	
124095	-0.484040	-0.667111	-0.415151	0.007236	-0.663649	1.641422	1.798427	2.620935	1.666756	1.687144	...	
124096	0.727384	1.802477	1.441911	-0.664373	1.348978	0.619656	0.701177	0.679178	1.235757	0.955552	...	
124097	-0.080232	-0.443905	0.203869	-0.576772	-0.609835	-0.139369	-0.490123	-0.577253	0.589259	0.223960	...	
124098	-0.484040	-0.309982	-0.415151	0.735299	-0.172152	-1.190328	-1.148472	-0.977026	1.235757	0.955552	...	

124099 rows x 47 columns

```
In [9]: class CustomDataset(Dataset):
def __init__(self, X, y, category):
    self.X = torch.tensor(X.values, dtype=torch.float32)
    self.y = torch.tensor(y.values, dtype=torch.long).unsqueeze(1)
    self.category = torch.tensor(category.values, dtype=torch.long)

def __len__(self):
    return len(self.X)

def __getitem__(self, idx):
    return self.X[idx], self.y[idx], self.category[idx]

train_dataset = CustomDataset(X_train, y_train, X_train_store_category_le)
val_dataset = CustomDataset(X_val, y_val, X_val_store_category_le)
test_dataset = CustomDataset(X_test, y_test, X_test_store_category_le)

batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
In [10]: for X_batch, y_batch, category_batch in val_loader:
print(f'X_batch shape: {X_batch.shape}, y_batch shape: {y_batch.shape}, category_batch shape: {category_batch.shape}')
break
```

X_batch shape: torch.Size([64, 47]), y_batch shape: torch.Size([64, 1]), category_batch shape: torch.Size([64])

```
In [11]: class Trainer:
def __init__(self, model, criterion, optimizer, log_dir=None):
    self.model = model.to(device)
    self.criterion = criterion
    self.optimizer = optimizer
    self.plotting_dict = {
        "train_loss": [],
        "val_loss": [],
    }
    self.early_stop_counter = 0
    self.early_stop_patience = 5
```

```
self.best_val_loss = float('inf')

# TensorBoard setup
if log_dir is None:
    current_time = datetime.now().strftime('%Y%m%d-%H%M%S')
    log_dir = os.path.join('runs', current_time)

self.writer = SummaryWriter(log_dir)

def train(self, train_dataloader, val_dataloader, epochs=5):
    self.model.train()
    dataset_size = len(train_dataloader)
    for epoch in range(epochs):
        total_loss = 0
        for i, (inputs, targets, category) in enumerate(train_dataloader):
            inputs, targets, category = inputs.to(device), targets.to(device), category.to(device)

            outputs = self.model(inputs, category)
            loss = self.criterion(outputs, targets.float())
            loss.backward()
            self.optimizer.step()
            self.optimizer.zero_grad()

            total_loss += loss.item()

        if i % 10 == 0:
            step = epoch * len(train_dataloader) + i
            self.writer.add_scalar('Batch/Loss', loss.item(), step)

    individual_loss = total_loss / dataset_size
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {individual_loss:.4f}")

    val_loss = self.evaluate(val_dataloader, "Validation")
    if val_loss < self.best_val_loss:
        self.best_val_loss = val_loss
        self.early_stop_counter = 0
    else:
        self.early_stop_counter += 1
        if self.early_stop_counter >= self.early_stop_patience:
            print(f"Early stopping at epoch {epoch}")
            break

    self.plotting_dict["train_loss"].append(individual_loss)
    self.plotting_dict["val_loss"].append(val_loss)

    self.writer.add_scalar('Loss/Train', individual_loss, epoch)
    self.writer.add_scalar('Loss/Validation', val_loss, epoch)

    for name, param in self.model.named_parameters():
        self.writer.add_histogram(f'Parameters/{name}', param, epoch)
    return val_loss

def evaluate(self, dataloader, mode="Evaluation"):
    self.model.eval()
    loss = 0
    with torch.no_grad():
        for inputs, targets, category in dataloader:
            inputs, targets, category = inputs.to(device), targets.to(device), category.to(device)
            outputs = self.model(inputs, category)
            loss += self.criterion(outputs, targets.float()).item()
    loss /= len(dataloader)
    print(f"{mode} Loss: {loss:.4f}")
    print("-----")

    return loss

def plot(self):
    fig = plt.figure(figsize=(10, 6))
    plt.plot(self.plotting_dict["train_loss"], label="Train Loss")
    plt.plot(self.plotting_dict["val_loss"], label="Validation Loss")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend()
    plt.show()
    plt.tight_layout()

# self.writer.add_figure('Loss Curves', fig)

return fig

def close(self):
    """Close the TensorBoard writer when done"""
    self.writer.close()
```

Simple Neural Network

```
In [17]: class SimpleNN(nn.Module):
def __init__(self, input_size):
    super().__init__()
    self.fc1 = nn.Linear(input_size, 256)
    self.fc2 = nn.Linear(256, 128)
    self.fc3 = nn.Linear(128, 64)
    self.fc4 = nn.Linear(64, 32)
    self.fc5 = nn.Linear(32, 1)
    self.dropout = nn.Dropout(0.2)

def forward(self, x, category):
    x = torch.tanh(self.fc1(x))
    # x = self.dropout(x)
    x = torch.tanh(self.fc2(x))
    x = self.dropout(x)
    x = torch.tanh(self.fc3(x))
    x = self.dropout(x)
    x = torch.tanh(self.fc4(x))
    x = self.fc5(x)
    return x

In [18]: model = SimpleNN(X_train.shape[1]).to(device)
print(summary(model))
```

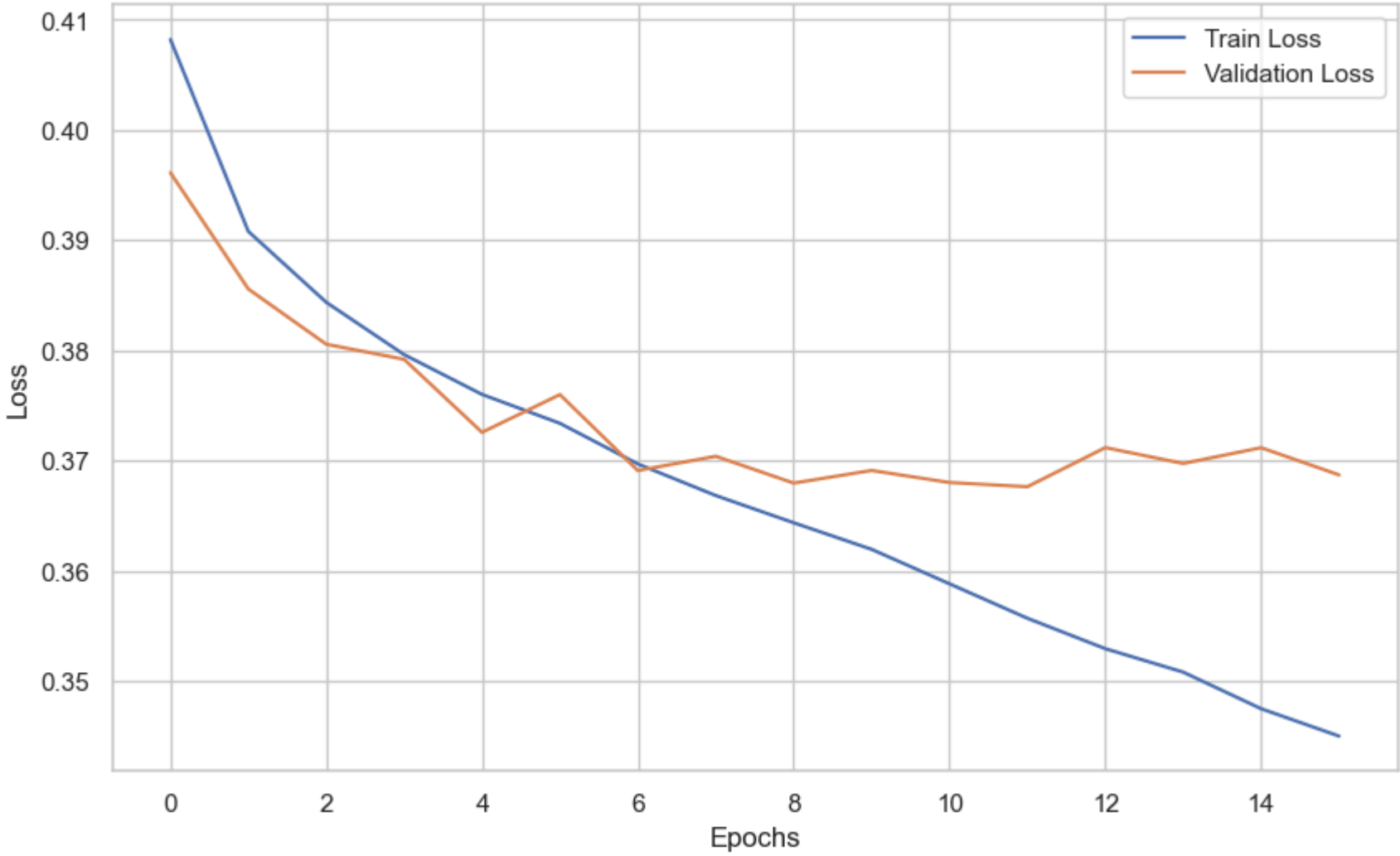
```
=====
Layer (type:depth-idx)              Param #
=====
SimpleNN                             --
├─Linear: 1-1                        12,288
├─Linear: 1-2                        32,896
├─Linear: 1-3                        8,256
├─Linear: 1-4                        2,080
├─Linear: 1-5                        33
└─Dropout: 1-6                       --
=====
Total params: 55,553
Trainable params: 55,553
Non-trainable params: 0
=====
```

```
In [19]: loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
In [20]: num_epochs = 20
         trainer = Trainer(model, loss_fn, optimizer)
         trainer.train(train_loader, val_loader, epochs=num_epochs)
         # trainer.evaluate(test_loader)
         send_notification("Training completed")

Epoch [1/20], Loss: 0.4082
Validation Loss: 0.3961
-----
Epoch [2/20], Loss: 0.3908
Validation Loss: 0.3856
-----
Epoch [3/20], Loss: 0.3844
Validation Loss: 0.3806
-----
Epoch [4/20], Loss: 0.3796
Validation Loss: 0.3792
-----
Epoch [5/20], Loss: 0.3760
Validation Loss: 0.3726
-----
Epoch [6/20], Loss: 0.3734
Validation Loss: 0.3760
-----
Epoch [7/20], Loss: 0.3697
Validation Loss: 0.3691
-----
Epoch [8/20], Loss: 0.3668
Validation Loss: 0.3704
-----
Epoch [9/20], Loss: 0.3644
Validation Loss: 0.3680
-----
Epoch [10/20], Loss: 0.3620
Validation Loss: 0.3691
-----
Epoch [11/20], Loss: 0.3588
Validation Loss: 0.3680
-----
Epoch [12/20], Loss: 0.3557
Validation Loss: 0.3676
-----
Epoch [13/20], Loss: 0.3529
Validation Loss: 0.3712
-----
Epoch [14/20], Loss: 0.3508
Validation Loss: 0.3697
-----
Epoch [15/20], Loss: 0.3475
Validation Loss: 0.3712
-----
Epoch [16/20], Loss: 0.3450
Validation Loss: 0.3687
-----
Epoch [17/20], Loss: 0.3424
Validation Loss: 0.3733
-----
Early stopping at epoch 16

In [21]: trainer.plot();
```



<Figure size 640x480 with 0 Axes>

Neural Network with Category Embedding Layer

```
In [26]: class NetworkWithCategoryEmbedding(nn.Module):
         def __init__(self, input_size, num_categories, embedding_dim):
             super().__init__()
             self.category_embedding = nn.Embedding(num_categories, embedding_dim)
             self.fc1 = nn.Linear(input_size, 256)
             self.fc2 = nn.Linear(256 + embedding_dim, 256)
             self.batchnorm_emb = nn.BatchNorm1d(embedding_dim)
             self.fc3 = nn.Linear(256, 128)
             self.fc4 = nn.Linear(128, 64)
             self.fc5 = nn.Linear(64, 32)
             self.fc6 = nn.Linear(32, 1)
             self.dropout = nn.Dropout(0.1)
             self.batchnorm = nn.BatchNorm1d(256)

         def forward(self, x, categories):
             emb = self.batchnorm_emb(self.category_embedding(categories))
             x = torch.relu(self.fc1(x))
             x = torch.cat((x, emb), dim=1)
             x = torch.relu(self.fc2(x))
             x = self.batchnorm(x)
             x = torch.relu(self.fc3(x))
             x = self.dropout(x)
             x = torch.relu(self.fc4(x))
             x = torch.relu(self.fc5(x))
             x = self.fc6(x)
             return x

In [28]: model_with_embedding = NetworkWithCategoryEmbedding(X_train.shape[1], len(X_train_store_category_le.unique()), 35).to(device)
         print(summary(model_with_embedding))
         optimizer = torch.optim.Adam(model_with_embedding.parameters(), lr=0.001)
         loss_fn = nn.MSELoss()
         # x_sample = torch.randn(1, X_train.shape[1])
```



```
# category_sample = torch.randint(0, len(X_train_store_category_le.unique()), (1,))
# writer.add_graph(model_with_embedding, (x_sample, category_sample))
# writer.close()
# model_with_embedding = model_with_embedding.to(device)
```

Layer (type:depth-idx)	Param #
NetworkWithCategoryEmbedding	--
├─Embedding: 1-1	2,310
├─Linear: 1-2	12,288
├─Linear: 1-3	74,752
├─BatchNorm1d: 1-4	70
├─Linear: 1-5	32,896
├─Linear: 1-6	8,256
├─Linear: 1-7	2,080
├─Linear: 1-8	33
├─Dropout: 1-9	--
├─BatchNorm1d: 1-10	512
Total params: 133,197	
Trainable params: 133,197	
Non-trainable params: 0	

```
In [29]: num_epochs = 15
optimizer = torch.optim.Adam(model_with_embedding.parameters(), lr=0.001)
e_trainer = Trainer(model_with_embedding, loss_fn, optimizer)
e_trainer.train(train_loader, val_loader, epochs=num_epochs)
# e_trainer.evaluate(test_loader) # Uncommented to evaluate the model
send_notification("Training completed") # Uncommented to send notification
e_trainer.close() # Close the TensorBoard writer
```

Epoch [1/15], Loss: 0.3964
Validation Loss: 0.3801

Epoch [2/15], Loss: 0.3804
Validation Loss: 0.3759

Epoch [3/15], Loss: 0.3738
Validation Loss: 0.3691

Epoch [4/15], Loss: 0.3681
Validation Loss: 0.3648

Epoch [5/15], Loss: 0.3637
Validation Loss: 0.3648

Epoch [6/15], Loss: 0.3592
Validation Loss: 0.3645

Epoch [7/15], Loss: 0.3553
Validation Loss: 0.3620

Epoch [8/15], Loss: 0.3512
Validation Loss: 0.3662

Epoch [9/15], Loss: 0.3485
Validation Loss: 0.3651

Epoch [10/15], Loss: 0.3455
Validation Loss: 0.3686

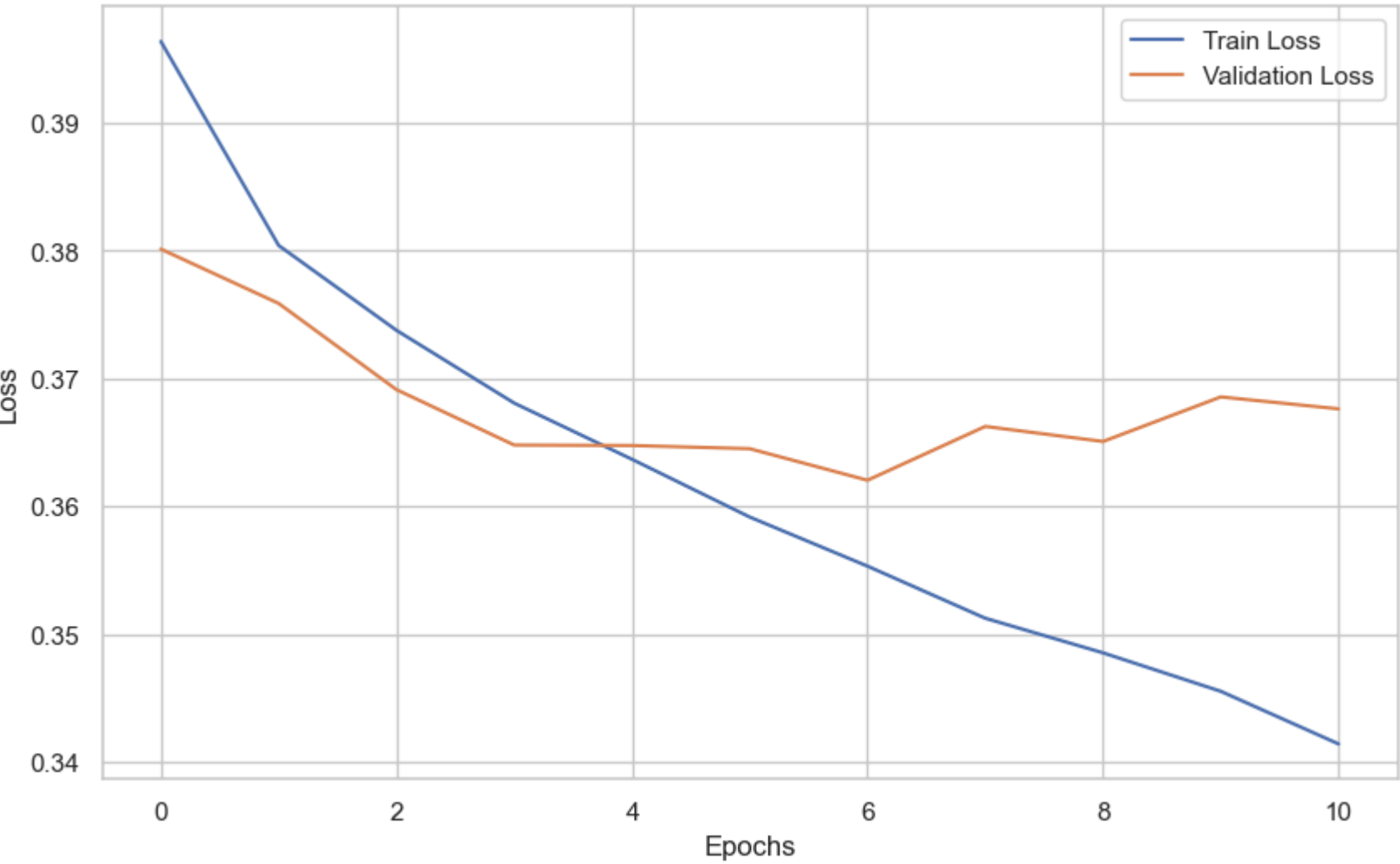
Epoch [11/15], Loss: 0.3414
Validation Loss: 0.3676

Epoch [12/15], Loss: 0.3376
Validation Loss: 0.3784

Early stopping at epoch 11

objective_cba40b48_19_activation=relu,layer1_neurons=128,layer2_neurons=256,layer3_neurons=128,layer4_neurons=16_2025-03-14_13-02-46',

```
In [30]: e_trainer.plot();
```



<Figure size 640x480 with 0 Axes>

Observations

- From above plot we can see that the model is overfitting the training data after epoch 6